

AI Modules

Intelligence Around the Fusion Core

PHASE 9 — RESEARCH-GRADE NOTES

Trinetra-AI

An Intelligent Multi-Sensor Fusion Framework for GPS-Denied Navigation

This is where Trinetra-AI earns the “AI”. Around the classical fusion core (Phases 7–8) sit learned modules that a Kalman filter alone cannot provide: **noise reduction, drift prediction, fault detection, sensor-health monitoring, anomaly detection**, and **confidence scoring** that feeds *adaptive* fusion. This document builds each module — the model, the mathematics, the training pipeline, the code, and the exact place it plugs into the navigation stack — always for processed multi-sensor data in *GPS-denied* conditions.

Intuition → model mathematics → training → code → integration with the fusion core.

Contents

1	Introduction	2
1.1	Why AI is needed in navigation	2
1.2	Why Kalman filters alone are insufficient (GPS-denied)	2
1.3	AI in navigation across domains	2
2	AI Architecture of Trinetra-AI	3
3	Noise Reduction Module	3
3.1	Why, and beyond classical filtering	3
3.2	Denoising autoencoder	4
4	Drift Prediction Module	5
4.1	The target	5
5	Fault Detection Module	6
6	Sensor Health Monitoring	6
7	Anomaly Detection	7
8	Confidence Scoring	8
8.1	Two kinds of uncertainty	8
8.2	Calibration	8
9	AI Models	9
10	Generative AI	11
10.1	Should Trinetra-AI use GANs? Where, exactly?	11
11	Reinforcement Learning	11
12	Explainable AI	12
13	Datasets	12
14	Research Papers	13
14.1	Foundational AI (used by the modules)	13
14.2	Navigation-AI (most relevant)	14
15	Complete AI Pipeline	15
16	Practical Implementation	15

17 Research Contribution	16
18 Interview & Research Preparation	16
18.1 70 interview questions	16
18.2 50 viva questions	18
18.3 40 research questions	20
18.4 30 coding questions	21
19 Summary, Glossary & Roadmap	21
19.1 Summary	21
19.2 Glossary	22
19.3 Roadmap: the complete Trinetra-AI architecture	22

1 Introduction

1.1 Why AI is needed in navigation

Classical fusion (the Kalman family) is *provably optimal* — but only under assumptions: *linear* dynamics, *Gaussian* noise with *known*, *stationary* covariances, and *correctly modelled* sensors with no faults. Real GPS-denied navigation violates every one of these. **AI modules** relax these assumptions by *learning* from data what the model does not know: nonlinear noise, time-varying drift, faults, and context-dependent sensor reliability.

KF assumption	Reality & the AI module that addresses it
Gaussian, known noise	structured/nonlinear noise → <i>learned denoising</i> (§3)
Stationary bias	time- & temperature-varying drift → <i>drift prediction</i> (§4)
No sensor faults	hard/soft/intermittent faults → <i>fault detection</i> (§5)
Fixed $\mathbf{R}$ per sensor	reliability varies with context → <i>confidence & adaptive weighting</i> (§8)
Model fully known	partial/unknown dynamics → <i>neural-Kalman, PINNs</i> (§9)

Intuition: the Kalman filter is a brilliant accountant who trusts the numbers you give it. The AI modules are the auditors who notice when a number is drifting, faulty, or untrustworthy — and adjust how much the accountant should believe it.

1.2 Why Kalman filters alone are insufficient (GPS-denied)

When GPS vanishes, there is no absolute reset for long stretches, so *every* unmodelled error accumulates. A fixed-gain filter cannot tell that a magnetometer just entered a distorted zone, that a gyro’s bias is trending with temperature, or that a measurement is an outlier — it weights them by static covariances. AI supplies the missing *context-awareness*: predict the drift before it corrupts the estimate, down-weight a sensor the moment it becomes unreliable, and report calibrated confidence for safety.

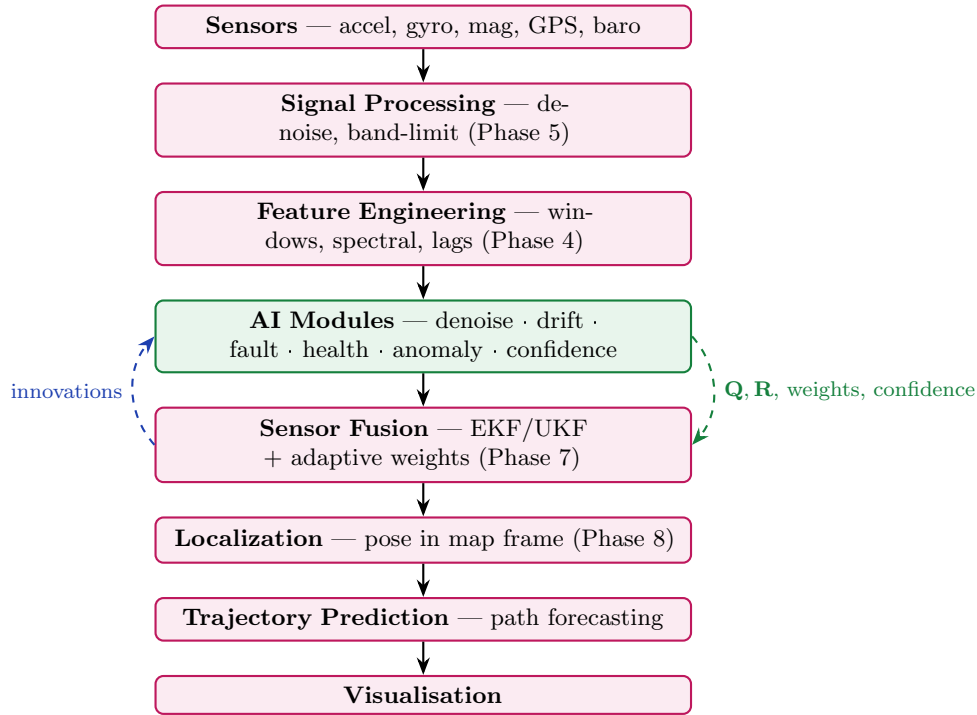
1.3 AI in navigation across domains

Domain	AI-for-navigation role
Robotics	learned odometry, anomaly-robust SLAM front-ends
Aerospace	MagNav de-noising (strip platform signature); health prognosis
Autonomous vehicles	learned displacement, adaptive GNSS trust in canyons
Defense	fault/spoof detection; passive-nav robustness
Space	fault-tolerant estimation with no ground truth
Quantum navigation	AI de-noising of quantum-magnetometer streams (the AQNav pattern)

The AI modules never replace the fusion core — they *wrap* it. This keeps the interpretability and guarantees of the Kalman/particle filter while adding the adaptivity that GPS-denied navigation demands. That hybrid — physics-structured core, data-driven modules — is Trinetra-AI’s central design principle and its research contribution.

2 AI Architecture of Trinetra-AI

The AI modules form a *layer* between feature engineering and fusion, plus a feedback path from the filter’s innovations.



Module	Input	Output → where it goes
Noise reduction (§3)	noisy sensor window	clean signal → fusion input
Drift prediction (§4)	IMU/temp window	bias forecast → pre-correct predict step
Fault detection (§5)	sensor features	fault flag → exclude/isolate sensor
Sensor health (§6)	long-term features	health index → weighting/-maintenance
Anomaly detection (§7)	measurement/innovation	anomaly score → gate outliers
Confidence (§8)	measurement + context	uncertainty → adaptive $\mathbf{R}$ /weights

Read the diagram as one loop: features flow *up* into the AI modules, which feed the fusion core the corrected measurements, adaptive noise, and weights; the filter’s *innovations* flow back *down* to drive fault/anomaly detection. This bidirectional coupling is what makes the fusion “intelligent” rather than fixed.

3 Noise Reduction Module

3.1 Why, and beyond classical filtering

Classical filters (Phase 5: Butterworth, Savitzky–Golay) remove *out-of-band*, *linear*, *stationary* noise cheaply. They fail on *in-band*, *nonlinear*, *signal-dependent* noise — e.g. vibration coupling that overlaps the motion band, or a magnetometer’s platform-signature. A learned denoiser handles exactly this structured residual.

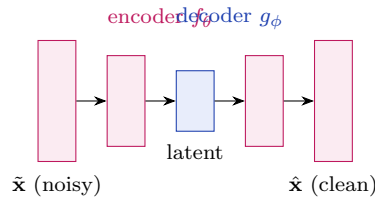
3.2 Denoising autoencoder

An **autoencoder** learns to reconstruct its input through a bottleneck: $\hat{\mathbf{x}} = g_\phi(f_\theta(\mathbf{x}))$, where the encoder f_θ compresses to a latent code and the decoder g_ϕ reconstructs. A **denoising autoencoder (DAE)** is trained to reconstruct the *clean* signal from a *corrupted* input $\tilde{\mathbf{x}}$, forcing it to learn robust structure rather than copy noise.

Corrupt the clean signal $\mathbf{x}$ (add sensor-like noise) to get $\tilde{\mathbf{x}}$, then minimise the reconstruction loss

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\mathbf{x}} \|\mathbf{x} - g_\phi(f_\theta(\tilde{\mathbf{x}}))\|_2^2.$$

The bottleneck prevents the identity map, so the network must learn the signal manifold. For sensor time series, f, g are 1-D convolutional or recurrent layers, giving a *sequence-to-sequence* denoiser. Brossard et al. (2020) use exactly this idea (a dilated CNN) to denoise gyroscopes for attitude estimation.



Denoiser	Strength	Trinetra-AI fit
CNN / TCN AE	fast, local patterns, parallel	real-time IMU denoise (primary)
LSTM/GRU AE	long temporal dependence	slow-drift-aware denoise
Transformer AE	long-range context	multi-sensor, high-data regime
Diffusion model	strong generative prior	offline high-quality denoise (heavy)

Training & evaluation. Loss: MSE (or Huber for robustness). Data: clean signals (bench/high-grade reference) with *synthetic* sensor noise, or paired low/high-grade recordings. Metrics: SNR improvement (dB), RMSE vs clean, and — the real test — downstream drift/ATE reduction after fusion.

```
import torch, torch.nn as nn
class ConvDAE(nn.Module): # 1-D denoising autoencoder for IMU
    def __init__(self, c=6):
        super().__init__()
        self.enc = nn.Sequential(nn.Conv1d(c,32,5,padding=2), nn.ReLU(),
                                nn.Conv1d(32,16,5,padding=2), nn.ReLU())
        self.dec = nn.Sequential(nn.Conv1d(16,32,5,padding=2), nn.ReLU(),
                                nn.Conv1d(32,c,5,padding=2))
        def forward(self, x): return self.dec(self.enc(x)) # x:(B,C,L) -> clean (B,C,L)
    loss = nn.MSELoss() # train: loss(model(noisy), clean)
```

The denoiser sits between filtering and fusion: it cleans the in-band residual that classical filters cannot, feeding sharper measurements to the EKF. Its highest-value target in Trinetra-AI is the **magnetometer's platform signature** — the core obstacle of magnetic

navigation — turning a noisy field reading into a usable map-matching measurement.

4 Drift Prediction Module

4.1 The target

Sensor drift is the slow, structured growth of bias over time (and with temperature). Because it is smooth and low-frequency, it is *forecastable* — the ideal regime for sequence models. Predicting the next-window bias and subtracting it *before* integration removes the dominant, fastest-growing inertial error.

Given a window of IMU (and temperature) features $\mathbf{X}_{t-L+1:t}$, learn $\hat{\mathbf{b}}_{t+1} = f_{\theta}(\mathbf{X}_{t-L+1:t})$ minimising $\mathcal{L} = \|\mathbf{b}_{t+1} - \hat{\mathbf{b}}_{t+1}\|^2$, where the target bias $\mathbf{b}$ comes from a reference (high-grade IMU, or filter-estimated bias). At run time, correct the raw reading: $\mathbf{z}_{\text{corr}} = \mathbf{z} - \hat{\mathbf{b}}$, shrinking the effective process noise $\mathbf{Q}$ the Kalman filter must carry.

Model	Why	Trade-off
Linear/ridge regression	baseline; temperature→bias	misses nonlinearity
LSTM / GRU	long temporal dependence	sequential, slower
TCN	parallel, deterministic latency	fixed receptive field
Transformer / Informer	long-range, multi-sensor	data-hungry

Training / evaluation / deployment. Train on windows with known bias; evaluate forecast error *vs. horizon* and, end-to-end, the reduction in dead-reckoning drift during simulated GPS outages. Deploy the light TCN/GRU on the edge (Jetson) for real-time pre-correction.

```
import torch, torch.nn as nn
class DriftTCN(nn.Module): # predict next-step bias from IMU window
    def __init__(self, c=6, out=6):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv1d(c,32,3,padding=2,dilation=2), nn.ReLU(),
            nn.Conv1d(32,32,3,padding=4,dilation=4), nn.ReLU(),
            nn.AdaptiveAvgPool1d(1), nn.Flatten(), nn.Linear(32,out))
    def forward(self, x): return self.net(x) # x:(B,C,L) -> bias (B,out)
```

This module is Trinetra-AI's proactive complement to the Kalman filter's *reactive* bias estimation: the filter corrects bias through innovations *after* it shows up; the drift network predicts and removes it *before* it corrupts the integration — attacking the t (gyro) and t^2 (accel) error terms that dominate a GPS outage.

5 Fault Detection Module

A **sensor fault** is a deviation from correct behaviour. **Hard faults** are abrupt/total (dropout, stuck value); **soft faults** are gradual/partial (growing bias, scale error); **intermittent faults** appear and vanish. **Detection** flags that *a* fault occurred; **isolation** identifies *which* sensor; **recovery** excludes or compensates it.

The workhorse is the **innovation** (residual) from the Kalman filter, $\mathbf{y} = \mathbf{z} - \mathbf{H}\hat{\mathbf{x}}^-$, with covariance $\mathbf{S}$. Under normal operation the normalised innovation squared (**NIS**) $\mathbf{y}^\top \mathbf{S}^{-1} \mathbf{y}$ follows a χ^2 distribution; a value above the χ^2 threshold flags a fault. Learned detectors generalise this: an autoencoder trained on healthy data flags a fault when its *reconstruction error* $\|\mathbf{z} - \hat{\mathbf{z}}\|$ exceeds a learned bound — catching soft/nonlinear faults the χ^2 test misses.

Method	Idea	Fault type caught
χ^2 / NIS gate	residual too large	hard, large soft
Autoencoder residual	reconstruction error	soft, nonlinear
One-class SVM / isolation forest	learn healthy region	novel/unseen faults
GNN	sensors as a graph; relational consistency	isolation across correlated sensors
Bayesian / GP	probabilistic residual	uncertainty-aware flags

Fault detection gives Trinetra-AI **graceful degradation**: when a sensor faults (e.g. a magnetometer near a large ferrous mass), it is detected, isolated, and down-weighted or excluded, and the filter continues on the remaining sensors. **Explainable AI** (§12) matters here — an engineer must be able to see *why* a sensor was flagged before trusting an automatic exclusion in a safety-critical system.

6 Sensor Health Monitoring

Health monitoring tracks the *gradual* condition of a sensor over its lifetime, beyond momentary faults. A **health index** summarises reliability (0–1); **Remaining Useful Life (RUL)** forecasts time-to-failure; **predictive maintenance** acts before failure. Monitoring can be *online* (live) or *offline* (post-analysis).

Track features that trend with degradation — noise-floor level, bias-drift rate, innovation magnitude, autocorrelation decay — and map them to a scalar health $h \in [0, 1]$, e.g. via a learned regressor $h = \sigma(\mathbf{w}^\top \phi(\text{features}))$ trained on run-to-failure data. RUL is a forecasting problem: predict the horizon until h crosses a failure threshold, using an LSTM over the health trajectory.

Approach	Method	Use
Feature + ML regressor	XGBoost/RF on degradation features	health index (interpretable)
LSTM RUL	forecast degradation trajectory	remaining useful life
Autoencoder drift	reconstruction error trend	unsupervised health

Health monitoring feeds the *adaptive weighting*: a degrading sensor’s health index smoothly lowers its fusion weight (raises its **R**) *before* it fully faults — a softer, earlier intervention than hard fault exclusion. It also enables predictive maintenance for a deployed platform. Together, fault detection (abrupt) and health monitoring (gradual) cover the full spectrum of sensor degradation.

7 Anomaly Detection

An **anomaly** is a measurement that deviates from expected behaviour. **Point** anomalies are single outliers (a GPS multipath jump); **contextual** anomalies are normal-valued but wrong-in-context (a large acceleration while “stationary”); **collective** anomalies are abnormal *patterns* over a window. Detection is usually *unsupervised* — anomalies are rare and unlabelled.

Method	Principle	Anomaly type
Isolation Forest	random splits isolate outliers fast	point (Liu et al., 2008)
One-Class SVM	learn a boundary around normal data	point/novelty
Local Outlier Factor	local density deviation	point (local)
Autoencoder	high reconstruction error	point/collective
VAE	low likelihood under learned density	probabilistic
Transformer / forecast residual	large prediction error	contextual/temporal
GAN (AnoGAN-style)	poor generative fit	complex/collective

Two dominant scores: (1) **reconstruction error** of an autoencoder trained on normal data, $s = \|\mathbf{z} - g(f(\mathbf{z}))\|$ — high s = anomaly; (2) **Mahalanobis distance** of a feature vector, $D^2 = (\boldsymbol{\phi} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{\phi} - \boldsymbol{\mu})$, thresholded against a χ^2 quantile. Both convert “how surprising is this measurement?” into a scalar the fusion filter can gate on.

Evaluation. Precision/recall and PR-AUC (accuracy is misleading under heavy class imbalance); for time series, point-adjusted F1. Tune the threshold on a validation set with known injected anomalies.

```
from sklearn.ensemble import IsolationForest
import numpy as np
det = IsolationForest(contamination=0.02, random_state=0).fit(features_train)
scores = -det.score_samples(features_live) # higher = more anomalous
flag = scores > np.quantile(scores, 0.98) # gate the top 2% as anomalies
```

Anomaly detection gates outliers *before* they enter the Kalman filter — essential for GPS multipath jumps (non-Gaussian, KF-breaking) and magnetometer spikes. It shares machinery with fault detection but operates at the *measurement* level per update, whereas fault detection judges a *sensor* over time. The two together protect the fusion core from bad data.

8 Confidence Scoring

Confidence is the module that turns AI outputs into something the Kalman filter can *use*: a per-measurement uncertainty that becomes the adaptive **R**.

8.1 Two kinds of uncertainty

Aleatoric uncertainty is irreducible noise in the data (sensor noise) — it does not shrink with more data. **Epistemic** uncertainty is the model’s own ignorance (unseen conditions) — it *does* shrink with more data. Navigation needs both: aleatoric sets how noisy a good measurement is; epistemic warns when the model is out of its training distribution (e.g. a novel magnetic environment).

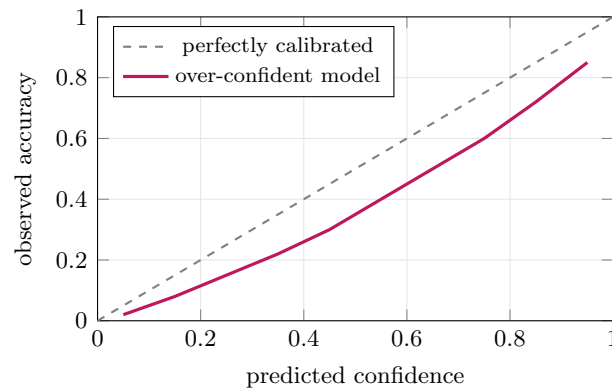
Aleatoric: train the network to output a *variance* as well as a mean, via the heteroscedastic Gaussian negative-log-likelihood

$$\mathcal{L} = \frac{\|\mathbf{y} - \boldsymbol{\mu}_{\theta}(\mathbf{x})\|^2}{2\sigma_{\theta}(\mathbf{x})^2} + \frac{1}{2} \log \sigma_{\theta}(\mathbf{x})^2,$$

so the model learns to say “this input is noisy”. **Epistemic**: approximate a Bayesian posterior over weights. **MC Dropout** (Gal & Ghahramani, 2016) keeps dropout *on* at inference and runs T stochastic passes; the predictive variance $\frac{1}{T} \sum_t (\hat{\mathbf{y}}_t - \bar{\mathbf{y}})^2$ estimates epistemic uncertainty. **Deep ensembles** (Lakshminarayanan et al., 2017) train M models and use their disagreement — often the strongest, simplest estimate.

8.2 Calibration

A model is **calibrated** if its stated confidence matches reality — of all predictions made with 90% confidence, $\approx 90\%$ are correct. Miscalibration is dangerous in navigation: an over-confident wrong measurement corrupts the filter. **Expected Calibration Error (ECE)** and **reliability diagrams** measure it; temperature scaling fixes it.



The over-confident curve sits *below* the diagonal — it claims more certainty than it delivers. Calibrating it is what makes the confidence score safe to feed the filter as $\mathbf{R}$.

How confidence improves navigation: the confidence module outputs a per-measurement uncertainty that becomes the filter’s *adaptive* $\mathbf{R}$ and per-sensor weight — the learned generalisation of the fixed covariances from Phase 6. A measurement the model is unsure about (high epistemic, e.g. a novel magnetic zone) is automatically down-weighted; a confidently clean one is trusted. *Calibration is non-negotiable:* an over-confident module would tell the filter to trust bad data, which is worse than no AI at all.

9 AI Models

A reference catalogue, judged specifically for Trinetra-AI’s needs (time-series, small-to-medium data, edge deployment, need for uncertainty).

Model	Core idea	Complexity	Suitability
Linear/Ridge	linear map	very low	baseline; temp→bias
Random Forest	bagged trees	low	health/fault (tabular)
XGBoost/LightGBM/CatBoost	gradient boosted trees	low	strong tabular baseline
SVM / One-Class SVM	margin / boundary	med	small-data / novelty
KNN	nearest neighbours	low (infer high)	quick baseline
MLP	fully connected net	low	feature→scalar heads
CNN / TCN	(dilated) convolutions	med	denoise, drift (real-time)
RNN	recurrence	med	baseline sequence
LSTM / GRU	gated memory	med	drift, trajectory
Transformer	self-attention	high	long-range multi-sensor
Informer	sparse attention	high	long-horizon forecasting
Temporal Fusion Transformer	attention + gating + quantiles	high	interpretable multi-horizon
Graph Neural Network	message passing on a graph	med-high	relational sensor fusion
Bayesian network / BNN	probabilistic	med-high	uncertainty-aware
Neural Kalman (KalmanNet)	learned gain in KF structure	med	hybrid fusion (headline)
Physics-Informed NN	physics residual in loss	med	enforce kinematics

Neural Kalman (KalmanNet) keeps the KF predict/update but replaces the gain $\mathbf{K} = \mathbf{P}^{-}\mathbf{H}^{\top}\mathbf{S}^{-1}$ with a recurrent network learning $\mathbf{K}$ from data — coping with unknown noise/model mismatch while staying interpretable. **Physics-Informed NN (PINN)** (Raissi et al., 2019) adds the governing equations as a *loss* term: for navigation, penalise violations of kinematics $\mathcal{L}_{\text{phys}} = \|\dot{\mathbf{p}} - \mathbf{v}\|^2 + \|\dot{\mathbf{v}} - (\mathbf{R}\mathbf{a} - \mathbf{g})\|^2$, so the network’s outputs obey physics even where data is sparse — data-efficient and trustworthy.

Trinetra-AI’s model choices: **TCN/GRU** for real-time denoise & drift (edge-friendly); **gradient-boosted trees** for interpretable tabular health/fault baselines; **Transformers** where long-range and data allow; **KalmanNet-style neural-Kalman** and **PINNs** as the research-grade hybrid fusion contributions. Start simple (trees, TCN), escalate to Transformers/neural-Kalman only when the data and the need justify it.

10 Generative AI

Generative models learn the data *distribution* $p(\mathbf{x})$ so they can *synthesise* new samples. **GAN**: a generator vs. discriminator minimax game. **Conditional GAN** / **TimeGAN** / **CycleGAN**: condition on labels / generate realistic time series / translate between domains. **VAE**: probabilistic autoencoder with a latent prior. **Diffusion**: iteratively denoise from noise. **Normalizing flow**: invertible exact-likelihood model.

GAN (Goodfellow et al., 2014): $\min_G \max_D \mathbb{E}[\log D(\mathbf{x})] + \mathbb{E}[\log(1 - D(G(\mathbf{z})))]$. **VAE** (Kingma & Welling, 2013) maximises the evidence lower bound $\mathcal{L} = \mathbb{E}_q[\log p(\mathbf{x} | \mathbf{z})] - \text{KL}(q(\mathbf{z} | \mathbf{x}) || p(\mathbf{z}))$, using the reparameterisation $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$. **Diffusion** (Ho et al., 2020) trains a network to predict the noise added at each step, minimising $\mathbb{E}\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\|^2$.

10.1 Should Trinetra-AI use GANs? Where, exactly?

Use case	Value	Verdict
Data augmentation	synthesise rare maneuvers/trajectories	yes (TimeGAN)
Synthetic sensor data	pre-train when real data scarce	yes (with care)
Missing-data generation	impute dropped GPS/sensor gaps	yes (VAE/diffusion)
Noise & fault simulation	generate realistic faults for training detectors	yes
Domain adaptation	sim→real, platform→platform	yes (CycleGAN)
The estimator itself	replace the fusion core	no

Verdict: yes, as a supporting track — not the core. Generative models earn their place in Trinetra-AI for *augmentation* (synthesise rare GPS-outage maneuvers and realistic faults to train the detection modules), *imputation* (fill missing-sensor gaps), and *domain adaptation* (sim→real). They should *not* be the estimator — the fusion core must stay physics-based and interpretable. Treat GAN/VAE/diffusion as a data-engineering and robustness aid, prioritised *below* the denoise/drift/fault/confidence modules that directly improve navigation.

11 Reinforcement Learning

Reinforcement learning (RL) learns a **policy** $\pi(a | s)$ mapping **states** to **actions** to maximise cumulative **reward**. Formalised as a Markov Decision Process $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$; the **value function** $V^\pi(s) = \mathbb{E}[\sum_t \gamma^t r_t]$ and action-value $Q^\pi(s, a)$ obey the Bellman equation. Algorithms: **DQN** (value-based), **PPO** (policy-gradient, clipped), **SAC/DDPG** (continuous control).

RL is for *sequential decisions*, not point estimation. In Trinetra-AI the natural formulation is **adaptive sensor management**: state = current sensor health/confidence/innovation; action = per-sensor weights or fusion-mode choice; reward = $-(\text{navigation error}) - (\text{compute cost})$. A policy π_θ then learns *when to trust or switch sensors* — an *adaptive Kalman* controller. This is promising but harder to train and less sample-efficient than supervised confidence estimation.

Verdict: RL is a *future/optional* direction for Trinetra-AI, best suited to policy-level decisions (adaptive weighting, active sensing, mode-switching) rather than the estimator. For the thesis, supervised confidence-driven adaptive weighting (§8) is the lower-risk, higher-yield path; RL-based adaptive fusion is a compelling extension if time and data allow, with a supervised baseline for comparison.

12 Explainable AI

Explainable AI (XAI) makes model decisions interpretable to humans. In safety-critical navigation, an engineer must understand *why* a sensor was flagged or a measurement down-weighted before trusting the automation. Key tools: **SHAP** (Shapley feature attributions), **LIME** (local surrogate models), **attention visualisation**, **integrated gradients**, and **counterfactuals**.

Method	Explains	Navigation use
SHAP	per-feature contribution (game theory)	why a fault/health score fired
LIME	local linear surrogate	single-decision explanation
Attention viz	which inputs the model weighted	which sensor/time drove a prediction
Integrated gradients	input-attribution for nets	drift/denoise attribution
Counterfactuals	“what would flip the decision”	fault-threshold understanding

XAI is what lets a navigation engineer *trust and debug* the AI modules: SHAP on the fault detector shows whether it flagged a magnetometer because of a genuine field distortion or a spurious feature; attention maps reveal which sensor a Transformer relied on. For a research thesis and any real deployment, explainability turns the AI modules from a black box into an auditable component — essential when the AI can override sensor trust in a safety-critical loop.

13 Datasets

Real, verified; confirm licence/download on the official page. Difficulty is relative to a solo project. (Many double as training data for several modules.)

Dataset	Sensors	Module fit	Diff.	Ground truth
RoNIN	IMU (phone)	drift, trajectory	Med	3D traj.
TLIO	IMU (headset)	drift, confidence	Med	VIO
OxIOD	IMU (accel/gyro/-mag)	denoise, drift	Easy–Med	Vicon/VI
EuRoC MAV	IMU + stereo	denoise, fusion	Med	Vicon/Leica
KITTI	IMU+GPS+LiDAR+cam	fusion, anomaly	Med	RTK/INS
MagPIE	IMU + magnetometer	denoise (mag), map-match	Med–Hard	cm-level
DAF-MIT AIA	aeromagnetic + INS	mag de-noising	Hard	flight ref.
MagNav				
NASA C-MAPSS	turbofan run-to-failure	RUL / health	Med	failure cycle
comma2k19	IMU+GNSS+CAN	fault, fusion	Med	GNSS+fusion
UCI HAR	phone IMU	pipeline practice	Easy	activity labels

Note. For *fault* and *anomaly* detection, inject synthetic faults (dropout, bias step, spikes) into a clean dataset when no labelled faults exist — the standard practice. NASA C-MAPSS is the canonical *RUL / health-prognosis* benchmark; the DAF-MIT AIA set is purpose-built for the magnetic-de-noising module.

14 Research Papers

Verified/canonical, correctly attributed. Priority: ★★★ read first, ★★ core, ★ as needed.

14.1 Foundational AI (used by the modules)

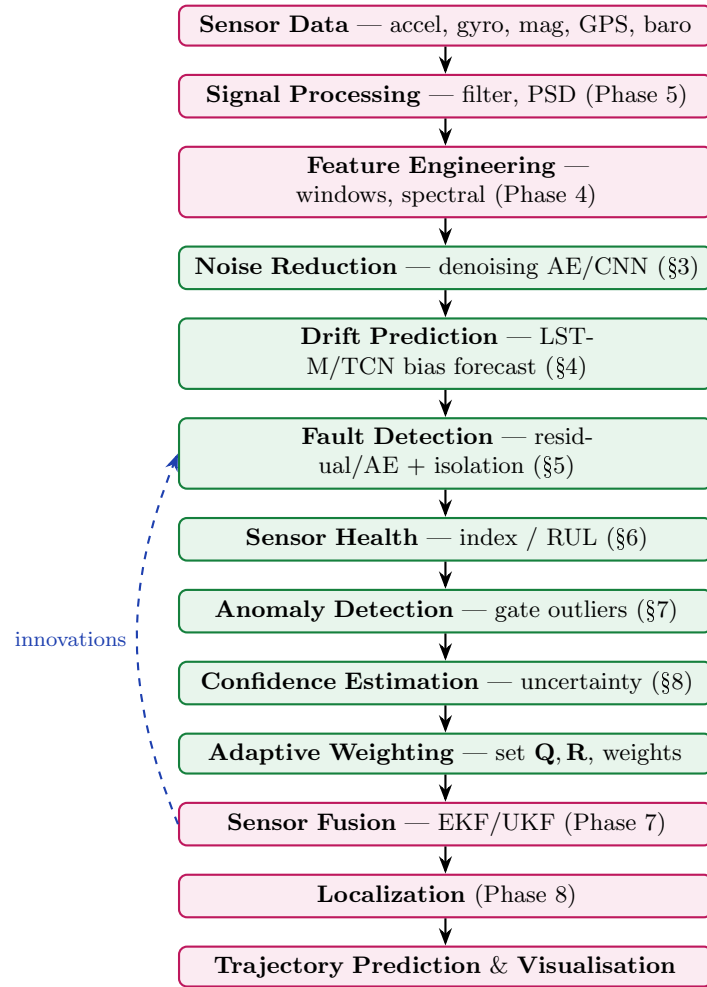
Work	Authors, year	Used for	Pri.
Long Short-Term Memory	Hochreiter & Schmidhuber, 1997	drift/trajectory	★★★
Attention Is All You Need	Vaswani et al., 2017	Transformer fusion	★★★
Generic Convolutional Nets (TCN)	Bai, Kolter & Koltun, 2018	denoise/drift	★★
Extracting Robust Features w/ Denoising Autoencoders	Vincent et al., 2008	denoising	★★
Auto-Encoding Variational Bayes (VAE)	Kingma & Welling, 2013	generative/anomaly	★★
Isolation Forest	Liu, Ting & Zhou, 2008	anomaly	★★
Dropout as Bayesian Approximation (MC Dropout)	Gal & Ghahramani, 2016	epistemic uncertainty	★★
Simple & Scalable Predictive Uncertainty (Deep Ensembles)	Lakshminarayanan et al., 2017	confidence	★★
A Unified Approach to Interpreting Predictions (SHAP)	Lundberg & Lee, 2017	explainability	★
Denoising Diffusion Probabilistic Models	Ho et al., 2020	generative	★
Physics-Informed Neural Networks	Raissi, Perdikaris & Karniadakis, 2019	physics-constrained nav	★★

14.2 Navigation-AI (most relevant)

Paper	Authors, year	Helps Trinetra-AI	Pri.
KalmanNet: NN-Aided Kalman Filtering	Revach et al., IEEE TSP 2022	neural-Kalman adaptive fusion	***
TLIO: Tight Learned Inertial Odometry	Liu et al., RA-L 2020	learned displacement + uncertainty	***
RoNIN	Herath et al., ICRA 2020	neural inertial nav + benchmark	***
IONet	Chen et al., AAAI 2018	first DNN inertial odometry	**
Denoising IMU Gyroscopes with DL	Brossard et al., RA-L 2020	the denoise module template	***
DAF-MIT AIA MagNav	Gnadt et al., 2023	magnetic de-noising benchmark	**
Quantum-assured magnetic navigation	Muradoglu et al., 2025	quantum-mag + AI denoising	**

Reading path: LSTM/TCN/Transformer foundations → Brossard (denoise) → RoNIN/TLIO (learned inertial) → KalmanNet (neural-Kalman) → Gal/Lakshminarayanan (uncertainty) covers every module in this phase. **Citation hygiene:** the AI foundations above are canonical — still verify DOIs before a report, and cite quantum-nav milestones from primary sources.

15 Complete AI Pipeline



Stage by stage. Cleaned, windowed features feed the AI modules *in sequence*: denoise the signal, predict and remove drift, screen for faults/anomalies, score health/confidence, and convert all of that into the *adaptive weights* and **Q**, **R** that drive the fusion core — whose innovations flow back to the fault/anomaly stages. The result is a navigation estimate that is not just fused but *context-aware*.

16 Practical Implementation

Notebook	Module	Expected output
01.denoise_ae.ipynb	denoising autoencoder (PyTorch)	clean signal; SNR gain
02.drift_tcn.ipynb	drift-prediction TCN/GRU	bias forecast; drift reduction
03.fault_detect.ipynb	residual/AE + isolation forest	fault flags; PR curves
04.health_rul.ipynb	health index + LSTM RUL (C-MAPSS)	health/RUL curves
05.anomaly.ipynb	AE/VAE + isolation forest	anomaly scores; PR-AUC
06.confidence.ipynb	MC-dropout / ensemble + calibration	uncertainty; reliability diagram
07.adaptive_fusion.ipynb	confidence → adaptive R in EKF	bounded GPS-denied trajectory
08.xai.ipynb	SHAP / attention on fault detector	feature attributions

Pipelines. *Train:* windowed features $\rightarrow$ model $\rightarrow$ loss (MSE/NLL/BCE), with early stopping on a *trajectory-split* validation set. *Validate/Test:* split by trajectory/time (never shuffle windows). *Deploy:* export to TorchScript/ONNX; run the light TCN/GRU on Jetson. **Metrics:** SNR/RMSE (denoise), forecast-error-vs-horizon (drift), PR-AUC (fault/anomaly), ECE/reliability (confidence), and end-to-end ATE/RTE & drift-rate after fusion. **Debugging:** watch for data leakage (shuffled overlapping windows), miscalibration (over-confident modules), and distribution shift. **Best practices:** strong classical baseline first; ablate each module; calibrate uncertainty; version data/configs/seeds; keep the fusion core physics-based. **Common mistakes:** training on moving data then testing on the same trajectory; optimising a module in isolation without checking downstream ATE; trusting an uncalibrated confidence.

17 Research Contribution

M.Tech-thesis-scale, publishable contributions: (1) *Confidence-driven adaptive fusion:* a calibrated uncertainty head that sets the EKF's **R** online, benchmarked against fixed and innovation-adaptive baselines on GPS-outage drift. (2) *AI-de-noised magnetic navigation:* learn to strip the platform's magnetic signature (the core MagNav problem) and quantify positioning gain on DAF-MIT AIA data. (3) *Neural-Kalman for magnetic-inertial navigation:* a KalmanNet-style learned gain under model mismatch, vs. a tuned EKF. (4) *Physics-informed drift/denoise:* enforce kinematic constraints in the loss for data efficiency and trustworthiness. (5) *Explainable fault-robust fusion:* SHAP-audited fault detection driving graceful degradation.

Research gaps & future work. Most learned-navigation work replaces *or* ignores the classical filter; Trinetra-AI's niche is the **hybrid** — a physics-based, interpretable core with calibrated, explainable AI modules *around* it, evaluated honestly under GPS denial with ablations. Future: integrate a (simulated then real) **quantum-magnetometer** channel and measure drift reduction; RL-based adaptive sensor management; on-device (edge) real-time operation; and on-line magnetic-map building (magnetic SLAM). *How AI improves classical fusion:* it relaxes the linear-Gaussian, known-noise, fault-free assumptions the KF cannot. *How AI improves quantum navigation:* de-noising is the enabling technology for reading nanotesla anomalies through platform interference — the exact problem AQNav/Ironstone Opal solve, and the one Trinetra-AI targets in software.

18 Interview & Research Preparation

18.1 70 interview questions

1. Why is AI needed in navigation?
2. Which KF assumptions break in reality?
3. Why are KFs insufficient GPS-denied?
4. AI modules wrap vs replace the filter?
5. What is an autoencoder?
6. Denoising autoencoder objective.
7. Why a bottleneck?
8. CNN vs LSTM vs Transformer denoiser.

9. In-band vs out-of-band noise.
10. Denoise loss functions.
11. Evaluate a denoiser.
12. What is sensor drift?
13. Why is drift forecastable?
14. Drift-prediction as regression.
15. LSTM vs TCN for drift.
16. Proactive vs reactive bias correction.
17. Hard vs soft vs intermittent fault.
18. Detection vs isolation vs recovery.
19. NIS/ χ^2 fault test.
20. Autoencoder fault detection.
21. GNN for fault isolation.
22. Health index vs fault.
23. What is RUL?
24. Predictive maintenance.
25. Online vs offline monitoring.
26. Point vs contextual vs collective anomaly.
27. Why unsupervised anomaly detection?
28. Isolation Forest.
29. One-Class SVM.
30. LOF.
31. Autoencoder vs VAE anomaly.
32. Reconstruction vs Mahalanobis score.
33. PR-AUC vs accuracy.
34. Aleatoric vs epistemic uncertainty.
35. Heteroscedastic NLL loss.
36. MC Dropout.
37. Deep ensembles.
38. What is calibration?
39. ECE / reliability diagram.
40. Temperature scaling.
41. Confidence $\rightarrow$ adaptive **R**.
42. Random forest vs XGBoost.
43. When trees over nets?
44. MLP vs CNN.
45. TCN receptive field.
46. LSTM gates.
47. Self-attention.

48. Informer.
49. Temporal Fusion Transformer.
50. GNN message passing.
51. Bayesian neural network.
52. KalmanNet idea.
53. Physics-informed NN loss.
54. GAN objective.
55. VAE ELBO.
56. Reparameterisation trick.
57. Diffusion training loss.
58. TimeGAN.
59. CycleGAN for domain adaptation.
60. Should Trinetra-AI use GANs?
61. RL MDP elements.
62. Value vs policy methods.
63. PPO vs DQN vs SAC.
64. RL for adaptive weighting.
65. Why XAI in navigation?
66. SHAP.
67. LIME.
68. Attention visualisation.
69. Integrated gradients.
70. Where each module plugs into fusion.

18.2 50 viva questions

1. Derive the DAE loss.
2. Derive the VAE ELBO.
3. Derive the reparameterisation trick.
4. Derive the heteroscedastic NLL.
5. Show MC dropout approximates a Bayesian posterior.
6. Derive the GAN minimax.
7. Derive the diffusion loss.
8. Set up drift forecasting formally.
9. Derive the NIS χ^2 test.
10. Reconstruction-error threshold choice.
11. Mahalanobis anomaly gate.
12. Calibrate with temperature scaling.
13. Compute ECE.
14. Explain KalmanNet gain learning.

15. Write the PINN kinematics loss.
16. Why physics loss aids data efficiency.
17. Isolation Forest mechanism.
18. One-Class SVM boundary.
19. Confidence $\rightarrow$ $\mathbf{R}$ mapping.
20. Adaptive weighting math.
21. Prevent leakage in evaluation.
22. Trajectory split rationale.
23. PR-AUC under imbalance.
24. RUL as forecasting.
25. Health-index regression.
26. Ensemble disagreement as epistemic.
27. Aleatoric variance head.
28. Why calibration matters for fusion.
29. Denoiser train data construction.
30. SNR-gain metric.
31. Fault isolation across correlated sensors.
32. GNN for sensor graphs.
33. Attention attribution.
34. SHAP additivity.
35. Counterfactual explanation.
36. RL reward design for nav.
37. Bellman equation.
38. PPO clipped objective.
39. When RL over supervised.
40. Diffusion vs GAN for imputation.
41. Domain adaptation $\text{sim} \rightarrow \text{real}$.
42. Ablate an AI module.
43. Downstream ATE as the true metric.
44. Distribution-shift detection.
45. Edge deployment constraints.
46. Model export (ONNX/TorchScript).
47. Overfitting signs.
48. Uncertainty propagation to fusion.
49. Failure modes of learned modules.
50. Explain the complete AI pipeline.

18.3 40 research questions

1. Calibrated confidence $\rightarrow$ adaptive **R**: measurable ATE gain?
2. Learned vs innovation-adaptive noise.
3. AI magnetic de-noising positioning gain.
4. Neural-Kalman vs tuned EKF under mismatch.
5. PINN drift model data efficiency.
6. Epistemic uncertainty for OOD magnetic zones.
7. Ensembles vs MC-dropout for nav confidence.
8. Fault-robust fusion vs classical gating.
9. RUL-driven predictive weighting.
10. Transformer vs TCN denoise trade-off.
11. GAN-augmented rare-maneuver robustness.
12. Diffusion imputation of sensor gaps.
13. CycleGAN sim $\rightarrow$ real transfer.
14. RL adaptive sensor management.
15. XAI-audited fault decisions.
16. GNN relational fusion gains.
17. Cross-platform generalisation.
18. Quantum-mag denoise ceiling.
19. Online magnetic-SLAM with learning.
20. Calibration under distribution shift.
21. Multi-module interaction effects.
22. Fusion stability when AI errs.
23. Confidence-triggered mode-switching.
24. Data-efficient hybrid vs end-to-end.
25. Sim-to-real gap quantification.
26. Anomaly-gating impact on ATE.
27. Long-horizon trajectory forecasting.
28. Uncertainty-aware map-matching.
29. Edge real-time constraints.
30. Federated health monitoring.
31. Physics-constrained confidence.
32. Attention as sensor-importance.
33. Robustness to injected faults.
34. Benchmark for GPS-denied AI navigation.
35. Explainability metrics for nav.
36. Active sensing policies.
37. Temporal-shift drift adaptation.
38. Multi-map (mag+terrain) learned fusion.
39. Safety guarantees for learned nav.
40. Reproducibility of learned-nav results.

18.4 30 coding questions

1. Build a 1-D denoising autoencoder (PyTorch).
2. Train it on noisy IMU windows.
3. Compute SNR gain.
4. Drift-prediction TCN.
5. Forecast-error-vs-horizon plot.
6. χ^2 fault gate from innovations.
7. Autoencoder fault detector.
8. Isolation Forest anomaly detector.
9. One-Class SVM.
10. VAE for anomaly.
11. PR-AUC evaluation.
12. Health-index regressor (XGBoost).
13. LSTM RUL on C-MAPSS.
14. MC-dropout uncertainty.
15. Deep ensemble uncertainty.
16. Heteroscedastic NLL head.
17. Reliability diagram + ECE.
18. Temperature scaling.
19. Confidence $\rightarrow$ adaptive $\mathbf{R}$ in an EKF.
20. KalmanNet-style gain net (toy).
21. PINN kinematics loss.
22. TimeGAN augmentation.
23. Conditional VAE imputation.
24. SHAP on a fault model.
25. Attention visualisation.
26. Integrated gradients.
27. Trajectory-split data loader.
28. Export to ONNX.
29. Inject faults & evaluate detection.
30. End-to-end ATE with vs without AI modules.

19 Summary, Glossary & Roadmap

19.1 Summary

The AI modules are the intelligence wrapped around Trinetra-AI's physics-based fusion core. Learned *denoising* cleans in-band noise classical filters miss; *drift prediction* proactively removes the fastest-growing inertial error; *fault detection* and *health monitoring* cover abrupt and gradual sensor degradation; *anomaly detection* gates bad measurements; and *confidence scoring* —

calibrated, separating aleatoric from epistemic uncertainty — converts everything into the *adaptive* $\mathbf{Q}, \mathbf{R}$ and per-sensor weights that make the Kalman filter context-aware. Generative models support augmentation, RL supports policy-level sensor management, and explainable AI keeps it all auditable. The guiding principle throughout: *hybrid* — a guaranteed, interpretable classical core, adapted by calibrated, explainable learned modules.

19.2 Glossary

Aleatoric/epistemic — data / model uncertainty. **Autoencoder (DAE)** — reconstructing (denoising) encoder-decoder. **Calibration / ECE** — confidence matching reality / its error. **Deep ensemble** — uncertainty from multiple models. **ELBO** — VAE training objective. **Health index / RUL** — reliability score / remaining useful life. **Isolation Forest / One-Class SVM / LOF** — unsupervised anomaly detectors. **KalmanNet** — neural-network-learned Kalman gain. **MC Dropout** — dropout at inference for Bayesian uncertainty. **NIS** — normalised innovation squared (χ^2 gate). **PINN** — physics-informed neural network. **SHAP/LIME** — feature-attribution explainability.

19.3 Roadmap: the complete Trinetra-AI architecture

Phase	Contribution to the whole
4 Time series	windowing, sequence models, temporal features
5 Signal processing	filtering, noise characterisation ($\mathbf{Q}, \mathbf{R}$)
6 Sensors	physics & error models; calibration
7 Sensor fusion	the Kalman/particle estimation core
8 Navigation	GPS-denied navigation system (INS + magnetic map + baro)
9 AI modules	the adaptive, calibrated, explainable intelligence around the core
Future	quantum-magnetometer channel; edge deployment; magnetic SLAM

The full picture. Trinetra-AI is a GPS-denied navigation framework whose classical core (fusion + navigation) is made *intelligent* by the AI modules of this phase and *quantum-ready* by the magnetic-navigation pipeline. Phases 4–9 form the complete software/AI stack: sense → process → model → fuse → navigate → adapt. The research contribution is precisely the *hybrid* — physics you can trust, wrapped in learning that adapts, with calibration and explainability that make it safe. That is the thesis, and the architecture is now complete.

End of Phase 9 notes — the AI modules complete the Trinetra-AI architecture. Future: quantum-magnetometer integration, edge deployment, and magnetic SLAM.